arXiv:2411.02886v4 [cs.CL] 9 Oct 2025

TokenSelect: Efficient Long-Context Inference and Length Extrapolation for LLMs via Dynamic Token-Level KV Cache Selection

Wei Wu[1†], Zhuoshi Pan[2†], Kun Fu[3], Chao Wang[1], Liyi Chen[4], Yunchu Bai[1], Tianfu Wang[5], Zheng Wang[3], Hui Xiong[5,6*]

[1]University of Science and Technology of China, [2]Tsinghua University, [3]Alibaba Cloud Computing, [4]Xiaohongshu Inc., [5]The Hong Kong University of Science and Technology (Guangzhou), [6]The Hong Kong University of Science and Technology

urara@mail.ustc.edu.cn, xionghui@ust.hk

Abstract

Rapid advances in Large Language Models (LLMs) have spurred demand for processing extended context sequences in contemporary applications. However, this progress faces two challenges: performance degradation due to sequence lengths out-of-distribution, and excessively long inference times caused by the quadratic computational complexity of attention. These issues limit LLMs in long-context scenarios. In this paper, we propose Dynamic Token-Level KV Cache Selection (*TokenSelect*), a training-free method for efficient and accurate long-context inference. *TokenSelect* builds upon the observation of non-contiguous attention sparsity, using QK dot products to measure per-head KV Cache criticality at token-level. By per-head soft voting mechanism, *TokenSelect* selectively involves a few critical KV cache tokens in attention calculation without sacrificing accuracy. To further accelerate *TokenSelect*, we design the Selection Cache based on observations of consecutive Query similarity and implemented the efficient Paged Dot Product Kernel, significantly reducing the selection overhead. A comprehensive evaluation of *TokenSelect* demonstrates up to 23.84× speedup in attention computation and up to 2.28× acceleration in end-to-end latency, while providing superior performance compared to state-of-the-art long-context inference methods.

1 Introduction

With the rapid development of large language models (LLMs), the number of parameters is no longer the sole factor significantly affecting model performance. The ability to effectively process longer context information has become one of the key metrics for evaluating LLMs' capabilities. Recent advances—such as cross-document understanding (Bai et al., 2024), LLM-powered search systems (Sharma et al., 2024), complex reasoning (OpenAI), and other cutting-edge LLM developments (Wu et al., 2025a; Pan et al., 2025; Wu et al., 2025b; Chen et al., 2024)—have all placed higher demands on the long-context capabilities of LLMs. There are two main difficulties in using pre-trained LLMs for long-context inference. On one hand, LLMs are limited by their context length during pre-training (*e.g.* Llama 3 only has 8192 tokens). Directly inferencing on longer sequences can lead to severe performance degradation due to reasons including sequence lengths out-of-distribution (Xiao et al., 2024b; Han et al., 2024). On the other hand, even if LLMs possess sufficiently large context lengths, the quadratic computational complexity of attention with respect to sequence length makes the response time for long-context inference unbearable.

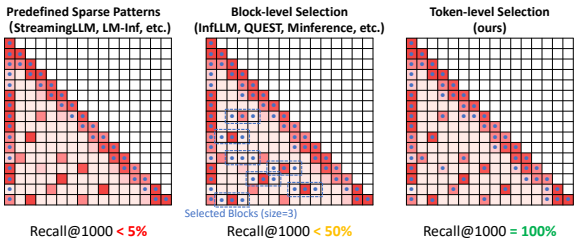

Figure 1: Distribution of tokens participating in attention computation under different sparsity patterns (blue dots. *TokenSelect* can more accurately select critical tokens (crimson squares) for attention computation.

Previous works have made numerous attempts to address these difficulties. To extend the context length of LLMs, the current common practice is to perform post-training on long texts (Yang et al., 2024a). However, this approach entails significant computational costs, motivating a training-free and effective method that is computationally efficient. To accelerate long-context inference, many stud-

*Corresponding Author.
†Equal Contribution.
§Code: https://github.com/pzs19/TokenSelect

ies focus on the sparsity of attention, attempting to reduce the scale of KV Cache involved in computation. The key to this type of method lies in designing sparse patterns for attention, which can be mainly divided into two categories: one uses predefined sparse patterns (Wang et al., 2019; Zaheer et al., 2020; Xiao et al., 2024b; Han et al., 2024), while the other estimates the potential importance of KV Cache during the inference process (Zhang et al., 2024b; Li et al., 2024; Lee et al., 2024; Tang et al., 2024; Jiang et al., 2024; Sun et al., 2025), attempting to select relevant KV Cache tokens into attention calculations. However, the design of these sparse patterns is often heuristically based on historical criticality or coarse-grained criticality estimation of tokens, making it difficult to ensure that the selected tokens are truly critical, thus resulting in sub-optimal performance, as shown in *Fig.* 1.

In this paper, we further observe the non-contiguous sparsity of attention, revealing the importance of designing more fine-grained dynamic sparse patterns. To this end, we propose *TokenSelect*, a training-free approach that utilizes token-level selective sparse attention for efficient long-context inference and length extrapolation. Specifically, for each Query, *TokenSelect* dynamically calculates token-level per-head criticality for the past KV Cache and selects the k most critical tokens through our head soft vote mechanism, involving them in the attention calculation. This reduces the scale of attention calculation to a constant length familiar to the model, while maintaining almost all of the long-context information, thereby simultaneously addressing the two main difficulties for long-context inference. To reduce the overhead of token selection, *TokenSelect* manages the KV Cache in token-level pages (Zheng et al., 2024) and design efficient kernel for token selection based on paged KV Cache management through Triton (Tillet et al., 2019). Furthermore, based on our observation of high similarity between consecutive queries, we have designed the Selection Cache, which allows consecutive similar queries to share token selection results, thereby reducing the selection frequency while ensuring its effectiveness.

We evaluate the performance and efficiency of *TokenSelect* on three representative long-context benchmarks using three open-source LLMs. The experimental results demonstrate that our *TokenSelect* can achieve up to $23.84\times$ speedup in attention computation compared to FlashInfer (flashinfer ai), and up to $2.28\times$ acceleration in end-to-end inference latency compared to state-of-the-art long-context inference method (Xiao et al., 2024a). Simultaneously, it provides superior performance on three long-text benchmarks. In summary, we make the following contributions:

- An observation on the non-contiguous sparsity of attention that highlights the importance of token-level KV Cache selection.
- *TokenSelect*, a training-free method that achieves accurate and efficient long-context inference and length extrapolation, which is compatible with mainstream LLM serving systems.
- Comprehensive evaluations of our method, showing up to $23.84\times$ speedup in attention computation and up to $2.28\times$ acceleration in end-to-end latency while exhibiting superior performance.

2 Related Works

Long-context LLMs. Due to computational complexity constraints, current LLMs based on Transformers often utilize limited context lengths during pre-training (Touvron et al., 2023; Dubey et al., 2024; Jiang et al., 2023; Yang et al., 2024a; GLM et al., 2024; AI et al., 2024). To extend their long-context capabilities, existing methods can be broadly categorized into three approaches (Huang et al., 2024; Zhou et al., 2024; Zhao et al., 2023): 1) Modifying positional encodings: A widely adopted method is positional interpolation (Chen et al., 2023). Chen et al. first proposed linear scaling of RoPE (Su et al., 2024) to map longer positional ranges within the original training window. Subsequent works (bloc97, 2023; emozilla, 2023) further improved this method using Neural Tangent Kernel (NTK) theory (Jacot et al., 2018), achieving longer context windows while maintaining model performance. Methods like YaRN (Peng et al., 2024) and Giraffe (Pal et al., 2023) optimize interpolation effects by adjusting frequency components or introducing temperature parameters. 2) Long-context post-training: This approach extends the model's context length through additional training steps on longer documents after pre-training (Yang et al., 2024b; Tian et al., 2025). It has been widely adopted by leading LLMs (Team et al., 2024; Yang et al., 2024a; GLM et al., 2024) with the support of sequence parallelism techniques (Shoeybi et al., 2020; Jacobs et al., 2024; Liu et al., 2024b). 3) Incorporating additional memory modules: Notable examples include Transformer-XL (Dai* et al., 2019), Com-

pressive Transformer (Rae et al., 2020), RMT (Bulatov et al., 2022) and Infini-attention (Munkhdalai et al., 2024). Although these methods have expanded the context length of LLMs, long-context inference still faces the challenge of high computational costs.

Efficient Long-context Inference. In state-of-the-art LLMs serving systems (Kwon et al., 2023; Zheng et al., 2024), technologies such as Flash Attention (Dao, 2024) and Paged Attention (Kwon et al., 2023) have greatly optimized LLMs inference efficiency by improving GPU I/O bottlenecks. However, in long-context inference scenarios, the computational complexity of attention poses new challenges for LLMs inference. Numerous studies focus on the sparsity of attention, selecting partial KV Cache for attention calculations to improve long-context inference efficiency. Sliding window (Wang et al., 2019; Zaheer et al., 2020) is one of the most widely used sparse patterns, reducing complexity to linear by executing attention computations within localized windows. Recent works like StreamingLLM (Xiao et al., 2024b) and LM-infinite (Han et al., 2024) retain the initial tokens of the sequence in addition to sliding windows, effectively maintaining LLMs' performance when processing long sequences. While these approaches are simple to implement, they cannot retain information from long contexts. Another approach focuses on KV Cache eviction during inference. Methods like H_2O (Zhang et al., 2024b), TOVA (Oren et al., 2024) and SnapKV (Li et al., 2024) evaluate token criticality based on historical attention scores, selecting tokens within a limited budget. However, these methods permanently discard parts of the KV Cache, causing information loss from long contexts. To address this, InfLLM (Xiao et al., 2024a) introduces Block Memory Units for KV Cache management, retrieving information from long contexts and offloading less-used blocks to CPU. Similarly, QUEST (Tang et al., 2024) proposes query-aware sparsity at page granularity, while MInference (Jiang et al., 2024) optimizes long-context inference using three sparse patterns. Apart from considering all attention heads, some other works (Ribar et al., 2024; Lee et al., 2024) attempt to focus on only a subset of attention heads. While existing methods have shown progress, opportunities for further improvement remain in achieving optimal accuracy and computational efficiency for real-world deployment.

3 Preliminaries

As discussed in the *Sec.* 1, the high attention sparsity in LLMs suggests sparse attention as a promising solution for long-context inference challenges, which can keep the number of tokens participating in attention computations at a constant scale. Given that predefined sparse patterns are detrimental to performance, we aim to dynamically select crucial tokens at each step during the inference process. Accordingly, based on the overview of LLM inference presented in Appendix D, we formalize the Selective Sparse Attention Problem as follows.

Definition 1 *(Selective Sparse Attention Problem, informal). For current input of length C ($C = 1$ in the decode stage) and KV Cache of length N, assuming there are H attention heads with size of d_h, let $\mathbf{O}$ be the output of the SDPA:*

$$\mathbf{O} = \left[\sigma\left(\frac{\mathbf{Q}^h \cdot [\mathbf{K}^h_{\text{cache}}, \mathbf{K}^h_{\text{current}}]^\top}{\sqrt{d}}\right) \cdot [\mathbf{V}^h_{\text{cache}}, \mathbf{V}^h_{\text{current}}]\right]_{h=1}^{H}, \tag{1}$$

where σ denotes softmax, $\mathbf{Q}^h, \mathbf{K}^h_{\text{current}}, \mathbf{V}^h_{\text{current}} \in \mathbb{R}^{C \times d_h}$ are Query, Key, Value matrices of current input for head h and $\mathbf{K}^h_{\text{cache}}, \mathbf{V}^h_{\text{cache}} \in \mathbb{R}^{N \times d_h}$ represent the KV Cache. Let $\hat{\mathbf{O}}$ be the output of the Selective Sparse Attention:

$$\hat{\mathbf{O}} = \left[\sigma\left(\frac{\mathbf{Q}^h \cdot [\mathbf{K}^h_{\text{select}}, \mathbf{K}^h_{\text{current}}]^\top}{\sqrt{d}}\right) \cdot [\mathbf{V}^h_{\text{select}}, \mathbf{V}^h_{\text{current}}]\right]_{h=1}^{H}, \tag{2}$$

where $\mathbf{K}^h_{\text{select}}, \mathbf{V}^h_{\text{select}} \in \mathbb{R}^{k \times d_h}$ are k selected KV Cache ($k \ll N$). The selection of $\mathbf{K}_{\text{select}}, \mathbf{V}_{\text{select}}$ is performed by selection function $\mathcal{S}$:

$$\begin{aligned} &\mathcal{S}(\mathbf{Q}, \mathbf{K}_{\text{cache}}) = \mathcal{I}, \text{ where } \mathcal{I} \in \mathcal{P}(\{1, \cdots, N\}), \\ &\mathbf{K}_{\text{select}} = [(\mathbf{K}_{\text{cache}})_i]_{i \in \mathcal{I}}, \ \mathbf{V}_{\text{select}} = [(\mathbf{V}_{\text{cache}})_i]_{i \in \mathcal{I}}, \end{aligned} \tag{3}$$

where $\mathcal{I}$ is the set of selected indices. The objective is to find an appropriate selection function $\mathcal{S}$ that minimizes the difference between the outputs of the SDPA and the selective sparse attention:

$$\min_{S} \left\| \mathbf{O} - \hat{\mathbf{O}} \right\|_2^2. \tag{4}$$

Existing works on long-context inference can be categorized under the Selective Sparse Attention Problem, with variations in the design of the selection function $\mathcal{S}$. Big-Bird and StreamLLM have developed input-independent selection functions $\mathcal{S}()$, while H_2O, TOVA and SnapKV propose Query-independent functions $\mathcal{S}(\mathbf{K}_{\text{cache}})$ for improved performance. Current state-of-the-art methods InfLLM, QUEST and MInference utilize Query-aware selection functions $\mathcal{S}(\mathbf{Q}, \mathbf{K}_{\text{cache}})$. However, these approaches typically select at a block-level, which limits their effectiveness.